

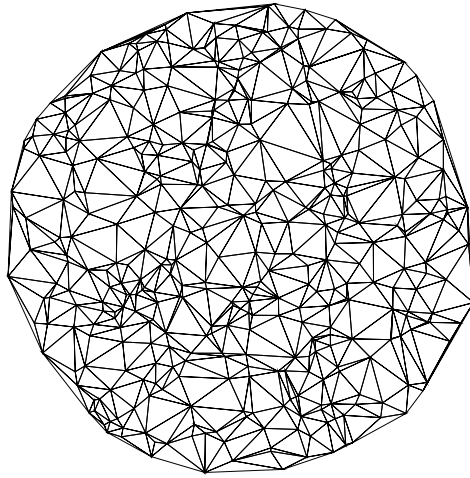


Figure 21.6.6. Delaunay triangulation of 300 points randomly chosen within a circle, as computed by the routines in this section.

21.7 Applications of Delaunay Triangulation

Emerging from the thicket of detail that was needed to implement the Delaunay triangulation, we are now ready to make use of it in several important applications. In this section we assume that you have a vector of points (say, `vecp`), and that you have invoked the code in §21.6 to construct a `Delaunay` structure. This usually means writing just the one line of code,

```
Delaunay mygrid(vecp);
```

So, what next?

21.7.1 Two-Dimensional Interpolation on an Irregular Grid

This is probably the most-asked-for algorithm that was missing from the previous two editions of *Numerical Recipes*. The basic setup is very simple. You are given a set of N points in the plane. You triangulate that set with a “good” triangulation, that is, one favoring short lines and big angles — in other words, Delaunay. You evaluate a function of interest at each of the points, and store the values in a vector (in the same order as the vector of points, of course).

Now it is easy to interpolate the function at a new point $\mathbf{p}$ that lies within the triangulation, that is, specifically, within the convex hull of your set of points. First, locate which triangle the point falls in. This takes only $O(\log N)$ operations if you use the `whichcontainspt()` method of the `Delaunay` structure. Then, linearly interpolate between the three function values at the three triangle vertices. The linear interpolation is uniquely defined, because (imagining your function plotted in the third dimension above the plane in which $\mathbf{p}$ lies) three points uniquely define a plane in three dimensions.

Constructively, the linear interpolation is easily done using barycentric coordinates as defined in equation (21.3.10), which in turn reduces to using the triangle area formula (equation 21.3.1) three times. Appropriately normalized, each barycentric coordinate value is exactly the weight of its corresponding vertex.

These ideas are implemented in the following function:

```

delaunay.h  Doub Delaunay::interpolate(const Point<2> &p,
const vector<Doub> &fnvals, Doub defaultval) {
Triangular grid interpolation of a function. Given an arbitrary point p and a vector of func-
tion values fnvals at the points that were used to construct the Delaunay structure, return
the linearly interpolated function value in the triangle in which p lies. If p lies outside of the
triangulation, instead return defaultval.
    Int n,i,j,k;
    Doub wgts[3];
    Int ipts[3];
    Doub sum, ans = 0.0;
    n = whichcontainspt(p);           Locate the point in the triangulation.
    if (n < 0) return defaultval;      Point outside of convex hull.
    for (i=0; i<3; i++) ipts[i] = thelist[n].p[i];
    for (i=0,j=1,k=2; i<3; i++,j++,k++) {   Calculate the barycentric coordinates, pro-
        if (j == 3) j=0;                   portional to the weights.
        if (k == 3) k=0;
        wgts[k]=(pts[ipts[j]].x[0]-pts[ipts[i]].x[0])*(p.x[1]-pts[ipts[i]].x[1])
            - (pts[ipts[j]].x[1]-pts[ipts[i]].x[1])*(p.x[0]-pts[ipts[i]].x[0]);
    }
    sum = wgts[0] + wgts[1] + wgts[2];      Normalization of the weights.
    if (sum == 0) throw("degenerate triangle");
    for (i=0; i<3; i++) ans += wgts[i]*fnvals[ipts[i]]/sum;   Linear interpolation.
    return ans;
}

```

Keep in mind that you should not expect high accuracy from linear interpolation. The interpolated function is piecewise linear, and continuous within the convex hull, but it has discontinuous derivatives in the direction perpendicular to the triangle's edges. On a triangle edge, it interpolates between the two function values at each end of the edge. You need a *lot* of triangles to get a reasonable representation of any function with much detailed structure.

21.7.2 Voronoi Diagrams

Around 1907, the Ukrainian mathematician Georgy Fedoseevich Voronoi revisited a problem that had been previously discussed by Dirichlet as early as 1850: Given N points, or *sites*, in the plane, each site $\mathbf{p}$ defines a region that is closer to $\mathbf{p}$ than to any of the other $N - 1$ sites. That region is called $\mathbf{p}$'s *Voronoi region*. What are the properties of the Voronoi regions, and how can we construct them?

If you imagine that everyone in a city shops at the closest supermarket ("as the crow flies"), then the Voronoi regions map out the districts served by each supermarket. If you imagine that fires are simultaneously set at sites in a forest, and that they spread circularly at a fixed speed, then the Voronoi regions are the areas burned by each different fire.

Figure 21.7.1, an example of a *Voronoi diagram*, shows the Voronoi regions around 40 sites chosen randomly in the plane. Yes, the boundaries of the Voronoi regions are polygons, although possibly open and extending to infinity. It is obvious, in fact, that the boundary of a site $\mathbf{p}$'s Voronoi region must consist of line segments each lying on the perpendicular bisector of the line connecting $\mathbf{p}$ to some other site, say $\mathbf{q}_i$. That is because the perpendicular bisector is the locus of points equidistant from $\mathbf{p}$ and $\mathbf{q}_i$. So the real questions are, for a given $\mathbf{p}$, which are the $\mathbf{q}_i$'s that contribute boundary segments? and is there a fast way to compute their intersections (the *vertices* of the Voronoi diagram)?

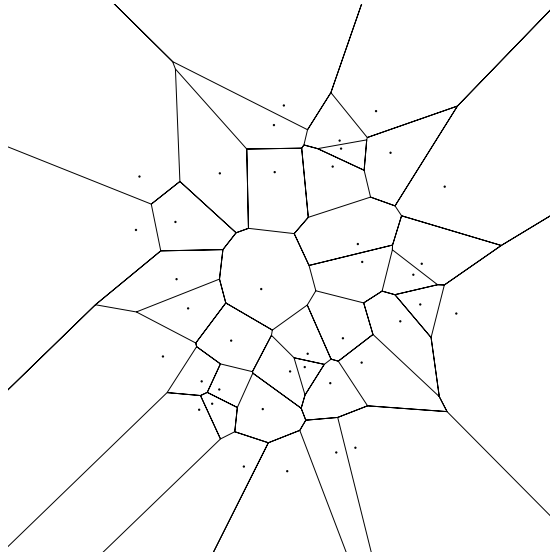


Figure 21.7.1. Voronoi diagram for 40 random sites. Each site has a Voronoi region, the area closer to it than to any other site. The boundaries of the Voronoi regions are straight-line segments that lie on the perpendicular bisectors between pairs of sites.

Remarkably, these questions are completely answered by the Delaunay triangulation of the Voronoi sites. (In fact, many texts start with the Voronoi diagram as fundamental, and then consider Delaunay triangulation as an application. We find it easier to go the other way.)

Some facts are

- Every edge in a site $\mathbf{p}$'s Voronoi region boundary lies on the perpendicular bisector of a Delaunay edge that connects to $\mathbf{p}$.
- In fact, every Delaunay edge corresponds to exactly one Voronoi edge, and vice versa.
- The vertices of the Voronoi diagram are exactly the circumcenters of the Delaunay triangles.
- The Voronoi diagram and the Delaunay triangulation are *dual graphs* (but don't worry if you don't know what this means)

Figure 21.7.2 shows the key ideas in the proof of the first two facts above. We already know that the boundary is made of *some* perpendicular bisector segments. We need to show that (i) every one of a point's Delaunay edges *does* contribute a segment, and (ii) lines drawn from that point to any other sites *don't* contribute any segments.

Part (a) of the figure shows a piece of Delaunay triangulation around site O . The perpendicular bisectors of OA and OC meet at the point X , which is therefore the center of the circle containing A , O , and C . The issue is whether the Delaunay edge OB can be “blocked” by the other two edges. Now, B must lie inside the circumcircle just mentioned; otherwise, the edge OB would have been an illegal edge when the Delaunay triangulation was constructed. But this means that the perpendicular bisector of OB , labeled UV , must “cut off the corner” at X . Thus it does contribute a segment to the boundary.

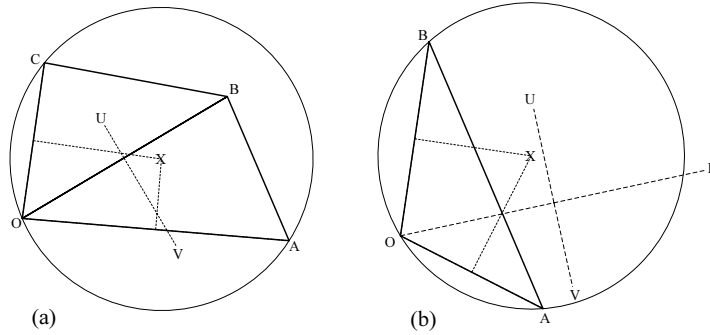


Figure 21.7.2. Key ideas in the proof that every Delaunay edge contributes exactly one Voronoi edge, lying on its perpendicular bisector. (a) Delaunay requires B to lie inside the circle AOC , hence its bisector must clip the corner inside X . (b) Delaunay requires any other site P to lie outside of the circle AOB , hence its bisector can't clip the corner inside X .

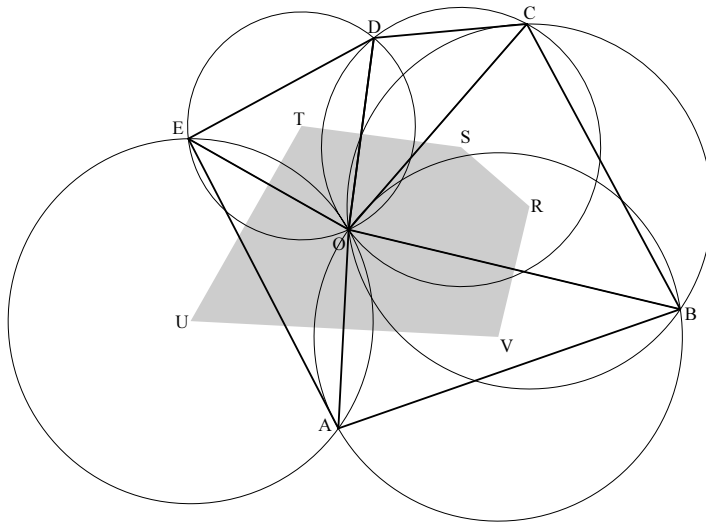


Figure 21.7.3. The circumcenters of the Delaunay triangles around a point O are the vertices of O 's Voronoi region (shown as shaded), because the perpendicular bisectors of the Delaunay edges meet at these circumcenters. Notice that a Voronoi edge need not actually intersect the Delaunay edge with which it is associated, as SR and OC .

Part (b) of the figure shows a Delaunay triangle OAB whose edges OA and OB are contributing perpendicular bisector segments to the Voronoi boundary around O . Point P is some other site. Can it somehow worm its way in close enough to contribute a segment of its own bisector, between the other two? Evidently not: We know that the circumcircle of any Delaunay triangle contains no other sites. Since P must lie outside the circumcircle, its bisector UV can't cut off the corner at X .

The fact that the Voronoi vertices are the circumcenters of Delaunay triangles is an immediate consequence of the previous discussion (see Figure 21.7.3). The circumcenters are where the perpendicular bisectors of the edges meet. Since every Delaunay edge contributes a segment, every such circumcenter must be a vertex. Notice, however, that not every Delaunay triangle contains its own circumcenter (as OCD in the figure), so that a segment on the boundary of a Voronoi region need not

actually intersect the Delaunay edge with which it is associated (as *RS* and *OC* in the figure).

We can count the number of edges and vertices in a Voronoi diagram with N sites (n of which are on the convex hull) simply by knowing that its dual is the Delaunay triangulation and using equation (21.6.1). The number of Voronoi edges is thus L in that equation, while the number of Voronoi vertices is T . The number of Voronoi regions is by definition N . The unbounded Voronoi regions are exactly those whose points lie on the convex hull of the sites, so there are n of these. It turns out (not immediately obviously) that the average number of edges in one Voronoi region (averaged over all the sites) does not exceed six.

Turning to the implementing code, it is convenient to have a structure for holding Voronoi edges, and also their association with the site that they surround (as an integer pointer to a list of sites).

```
struct Voredge {
    Point<2> p[2];
    Int nearpt;
    Voredge() {}
    Voredge(Point<2> pa, Point<2> pb, Int np) : nearpt(np) {
        p[0] = pa; p[1] = pb;
    }
};
```

voronoi.h

Structure for an edge in a Voronoi diagram, containing its two endpoints and an integer pointer to the site of which it is a boundary.

Now it is straightforward to define a Voronoi structure, as a derived class of the Delaunay structure. The constructor creates a Delaunay triangulation of the sites, and then loops over the sites. For each, it first finds any one triangle with the site as a vertex, and then works its way around the site in a circle, navigating counterclockwise from one triangle to the next by looking up their common edge in the `linehash` hash memory. Each triangle's circumcenter is a Voronoi vertex, and a Voronoi edge is stored for each two consecutive circumcenters as the site is circumnavigated.

```
struct Voronoi : Delaunay {
    Int nseg;
    VecInt trindx;
    vector<Voredge> segs;
    Voronoi(vector< Point<2> > pvec);
};
```

voronoi.h

Structure for creating a Voronoi diagram, derived from the Delaunay structure.

Number of edges in the diagram.

Will index triangles.

Will be array of all segments.

Construct Voronoi diagram from array of points.

```
Voronoi::Voronoi(vector< Point<2> > pvec) :
    Delaunay(pvec,1), nseg(0), trindx(npts), segs(6*npts+12) {
    Constructor for Voronoi diagram of a vector of sites pvec. Bit "1" sent to the Delaunay constructor tells it not to delete linehash.
    Int i,j,k,p,jfirst;
    Ullong key;
    Triel tt;
    Point<2> cc, ccp;
    for (j=0; j<ntree; j++) {
        if (thelist[j].stat <= 0) continue;
        tt = thelist[j];
        for (k=0; k<3; k++) trindx[tt.p[k]] = j;
    }
    for (p=0; p<npts; p++) {
        Create a table so that, given a point number, we can find one triangle with it as a vertex.
        Now loop over the sites.
```

```

tt = thelist[trindx[p]];
if (tt.p[0] == p) {i = tt.p[1]; j = tt.p[2];}
else if (tt.p[1] == p) {i = tt.p[2]; j = tt.p[0];}
else if (tt.p[2] == p) {i = tt.p[0]; j = tt.p[1];}
else throw("triangle should contain p");
jfirst = j;
ccp = circumcircle(pts[p],pts[i],pts[j]).center;
while (1) {
    key = hashfn.int64(i) - hashfn.int64(p);
    if ( ! linehash->get(key,k) ) throw("Delaunay is incomplete");
    cc = circumcircle(pts[p],pts[k],pts[i]).center;
    segs[nseg++] = Voredge(ccp,cc,p);
    if (k == jfirst) break;
    ccp = cc;
    j=i;
    i=k;
}
}
}

```

Get the vertices into canonical order.

Save starting vertex and its circumcircle.

Go around CCW, find circumcenters and store segments.

Circumnavigation completed. Normal way out.

The result of the Voronoi construct is available by looping through the `segs` array from 0 to `nseg-1`. Each array element is a `Voredge` that stores the endpoints, and also the site number with which it is associated. Note that each segment appears twice in the list, with opposite sense, as it is associated in turn with the sites on its two sides.

If you read our confession about dirty tricks in the previous section, you'll want to keep in mind that the "open" Voronoi polygons are actually closed by segments that lie at a distance of order `bigscale` times the size of the bounding box of the sites. Those segments are included in `segs` but appear only once, since there is no site on their other side.

21.7.3 Other Applications

Nearest Neighbors, Again. A line segment that connects a point to its nearest neighbor among a set of points will be an edge in the set's Delaunay triangulation. Informal proof: The nearest neighbor obviously must contribute a boundary to the Voronoi diagram. Formal proof (using a theorem mentioned above): The circle whose diameter connects a point to its nearest neighbor can't contain any other points (they'd be closer than the nearest neighbor), so that diameter must be a Delaunay edge.

Since we can construct the Delaunay triangulation in $O(N \log N)$ time, it follows that we can use it to find all nearest neighbors of a set of N points in $O(N \log N)$ time. The process is as follows: (i) Construct `Delaunay`. (ii) For each point, circumnavigate it. (We saw how to do this in our implementation of `Voronoi`, above.) (iii) Pick the shortest of the edges with the point at one end.

Convex Hull. Sometimes you may need to know the convex hull of a set of points in the plane for some other application. Although it might seem wasteful to construct the whole Delaunay triangulation just to get the hull, doing so is not too bad a method. Better efficiency can be achieved by ignoring, during the triangulation, points that are found to be already inside interior triangles. To sort the edges into the order of a CCW polygon, we create a `nexttpt` table of edge destinations as we go, then chain through it once to output the vertices of the convex hull in proper order.

delaunay.h

```

struct Convexhull : Delaunay {
    Structure for constructing the convex hull of a set of points in the plane. After construction,
    nhull is the number of points in the hull, and hullpoints[0..nhull-1] are integers pointing
    to points in the vector pvec that are in the hull, in CCW order.
    Int nhull;
    Int *hullpts;
    Convexhull(vector< Point<2> > pvec);           Construct from a vector of points.
};

Convexhull::Convexhull(vector< Point<2> > pvec) : Delaunay(pvec,2), nhull(0) {
    Constructor for convex hull of a vector of points pvec. Bit "2" sent to the Delaunay constructor
    tells it to ignore interior points when it can, for extra speed.
    Int i,j,k,pstart;
    vector<Int> nextpt(npts);
    for (j=0; j<ntree; j++) {
        if (thelist[j].stat != -1) continue;           Triangles with stat= -1 may contain
                                                         hull segments.
        for (i=0,k=1; i<3; i++,k++) {
            if (k == 3) k=0;
            if (thelist[j].p[i] < npts && thelist[j].p[k] < npts) break;
            Need two valid points to qualify.
        }
        if (i==3) continue;                           Case where failed to qualify.
        ++nhull;                                       Yes! Put its other end in the lookup table, and save
        nextpt[(pstart = thelist[j].p[k])] = thelist[j].p[i]; its value in case it's
                                                         the last one we find.
    }
    if (nhull == 0) throw("no hull segments found");
    hullpts = new Int[nhull];                         Now we know how many, can allocate.
    j=0;                                              One chain through the lookup table, start-
    i = hullpts[j++] = pstart;                       ing with pstart, gives the answer.
    while ((i=nextpt[i]) != pstart) hullpts[j++] = i;
}

```

Largest Empty Circle Problem. The largest empty circle whose center lies (strictly) inside the convex hull of a set of points in the plane has its center on a Voronoi vertex. So, you can find it by looping through the Voronoi vertices, calculating the radius of the largest circle centered on each one, and taking the maximum of these. Even better, loop through the Delaunay triangles, calculate the circumcenter of each, and pick the one with the largest circumradius (since Delaunay circumcenters are exactly Voronoi vertices). Think of yourself as finding the best location for a fast-food restaurant within the (convex) city limits, one that best avoids all the other fast-food restaurants.

Avoiding Obstacles. If you want to navigate around the plane staying as far as possible from a set of points, your path will be along the edges of a Voronoi diagram. Think of yourself as a fighter pilot avoiding enemy radars.

Minimum Spanning Tree. The *minimum spanning tree* (sometimes *Euclidean minimum spanning tree*) is the set of line segments of shortest total length that connect N points (see, e.g., Figure 21.7.4). Think of it as the highway map of the cheapest highway system that lets you visit all N cities. It is topologically a tree (i.e., has no loops) because if it did have a loop, you could save highway money by deleting one of the loop's segments.

The important theorem is: The minimum spanning tree is a subset of the Delaunay edges. You might think this isn't very useful, since it doesn't tell you *which* subset. Fortunately, there is a fast algorithm, *Kruskal's algorithm*, for doing the construction. The basic idea is to sort all the Delaunay edges by length, and then add them one at a time to the growing tree, in order from smallest to largest.

Your tree starts off growing in multiple disconnected components, but when

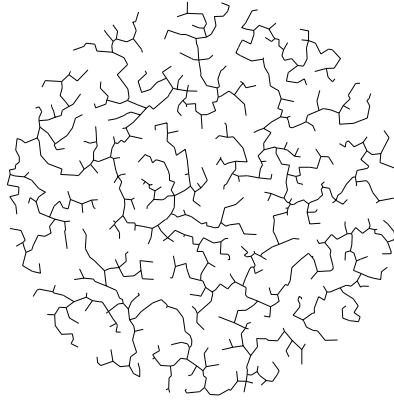


Figure 21.7.4. Minimum spanning tree of 1001 points at random locations within a circle. The tree is composed of 1000 segments that connect all the points with the shortest total length and is a subset of the Delaunay triangulation of the same points.

you have added exactly $N - 1$ segments, it will be a single piece, and the answer. There is only one catch: As you add segments, you must not add a segment if both its ends are already in the same component (else it would form a loop). So, you have to maintain an “equivalence class” relation for each vertex, making it equivalent to all other vertices in its connected component. We already know how to do this efficiently, as in §8.6’s routine `eclass`. In the code below, there is a similar logic of sweeping up pointers to single “mother” representatives. Do this properly, and the method is $O(N \log N)$.

Kruskal’s algorithm is a so-called *greedy algorithm*, since it just takes the best edge at each step willy nilly. It is rare for a greedy algorithm to yield the true global optimum; but this is the happy case where it does.

delaunay.h

```
struct Minspantree : Delaunay {
    Structure for constructing the minimum spanning tree of a set of points in the plane. After
    construction, nspan is the number of segments (always = npts-1), and minsega[0..nspan-1]
    and minsegb[0..nspan-1] contain integers pointing to points in the vector pvec that are the
    two ends of each segment.
    Int nspan;
    VecInt minsega, minsegb;           Allocate arrays for the output.
    Minspantree(vector< Point<2> > pvec);
};

Minspantree::Minspantree(vector< Point<2> > pvec) :
    Delaunay(pvec,0), nspan(npts-1), minsega(nspan), minsegb(nspan) {
    Constructor for the minimum spanning tree of a vector of points pvec. The Delaunay construc-
    tor gives the triangulation. We need only find the correct subset of edges.
    Int i,j,k,jj,kk,m,tmp,nline,n = 0;
    Tritel tt;
    nline = ntri + npts -1;           Number of edges in the triangulation.
    VecInt sega(nline);               Allocate working space for two ends of each edge,
    VecInt segb(nline);               edge length, and index on which we will sort.
    VecDoub segd(nline);              Also the "mother" tree for equivalence classes.
    VecInt mo(npts);
    for (j=0; j<ntree; j++) {         Find all edges in the triangulation, store them
        if (thelist[j].stat == 0) continue; and their lengths.
        tt = thelist[j];
        for (i=0,k=1; i<3; i++,k++) {
            if (k==3) k=0;
```